

Tutorial 05: Web Interactivity with JavaScript

Tutorial 05: Web Interactivity with JavaScript

Pemrograman Berbasis Platform (CSGE602022) - diselenggarakan oleh Fakultas Ilmu Komputer Universitas Indonesia, Semester Gasal 2026/2027

Kontributor: AYY - Farrel Arrayyan Adrianshah, FCB - Ahmad Faiq Fawwaz Abdussalam, CHAE - Muhammad Aldo Fahrezy

Tujuan Pembelajaran

Setelah menyelesaikan tutorial ini, mahasiswa diharapkan dapat:

- Memahami fungsi JavaScript pada *front-end development*
- Menggunakan JavaScript secara dasar
- Menerapkan AJAX dan Fetch API dengan aman

Peringatan

Tutorial ini adalah prasyarat wajib untuk Individual Assignment 5. Jika tutorial ini belum diselesaikan, Individual Assignment 5 tidak akan dinilai. Deadline Tutorial 05 adalah **Rabu, 30 September 2026**. Deadline Individual Assignment 5 adalah **Senin, 5 Oktober 2026**. Lihat halaman Jadwal untuk tanggal lengkap.

Yang Sudah Kita Bangun Sejauh Ini

Pada tutorial sebelumnya, kita telah menambahkan sistem autentikasi (registrasi, *login*, dan *logout*) serta mencatat waktu *login* menggunakan *cookie* pada proyek web portofolio. Kita juga telah menerapkan otorisasi hak akses. Hanya pemilik portofolio yang dapat mengelola proyek, sementara pengguna terdaftar lainnya hanya diizinkan memberikan *star*.

Tutorial ini adalah **Tutorial 05**, lanjutan langsung dari Tutorial 04, dalam rangkaian pembangunan **website portofolio pribadi** sepanjang semester. Gunakan proyek **myportofolio** yang sudah kamu kerjakan, dan pastikan fitur autentikasi, manajemen sesi, serta pembatasan hak akses (termasuk fitur *login*, *logout*, dan pemberian *star* pada proyek) telah berjalan dengan baik sebelum melanjutkan. Hasil tutorial ini akan menjadi dasar untuk tutorial berikutnya.

Sepanjang tutorial ini, contoh yang dipakai adalah portofolio milik **Burhan**, maskot mata kuliah PBP. Ganti data contoh pemilik portofolio, seperti nama, NPM, bio, foto, dan tautan sosial, dengan data kamu sendiri.

JavaScript

JavaScript merupakan bahasa pemrograman multiparadigma tingkat tinggi lintas platform (*cross-platform high-level multi-paradigm programming language*). Sifat multiparadigma membuat JavaScript mendukung konsep pemrograman berbasis objek, pemrograman imperatif, dan pemrograman fungsional. JavaScript sendiri adalah implementasi ECMAScript, standar yang mendefinisikan inti bahasa JavaScript. Beberapa implementasi lain dari ECMAScript yang mirip dengan JavaScript antara lain JScript (Microsoft) dan ActionScript (Adobe).

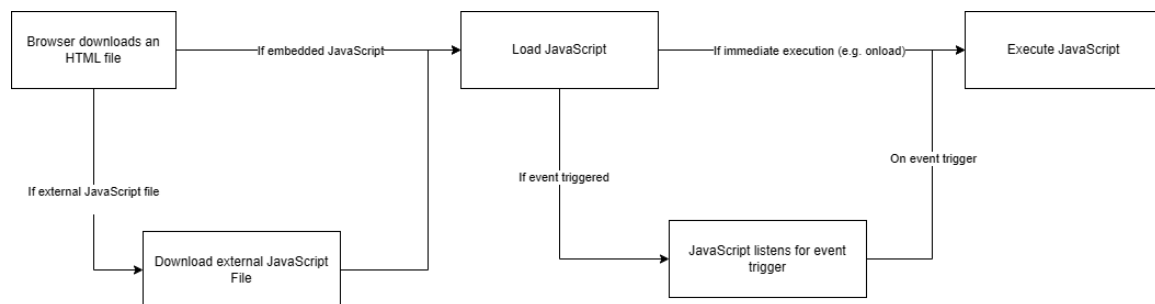
JavaScript, bersama dengan HTML dan CSS, menjadi tiga teknologi utama yang dipakai pada pengembangan web. Pada dasarnya, keuntungan menggunakan JavaScript dalam pengembangan web adalah manipulasi halaman web dapat dilakukan secara dinamis dan interaksi antara halaman web dengan pengguna dapat meningkat. Oleh karena itu, hampir semua situs web modern menggunakan JavaScript dalam halaman web mereka untuk memberikan pengalaman terbaik kepada pengguna. Beberapa contoh yang dapat kita lakukan dengan JavaScript antara lain menampilkan informasi berdasarkan waktu, mengenali jenis browser pengguna, melakukan validasi form atau data, membuat *cookie* (bukan kue, melainkan HTTP *cookie*), serta mengganti *styling* dan CSS suatu elemen secara dinamis.

Pada pengembangan web, kode JavaScript umumnya digunakan pada sisi klien suatu web (*client-side JavaScript*), tetapi JavaScript juga dapat dijalankan di sisi server (*server-side JavaScript*), misalnya dengan Node.js. Istilah *client-side* menunjukkan bahwa kode JavaScript dieksekusi pada browser pengguna, bukan pada server situs web. Hal ini berarti kompleksitas kode JavaScript tidak memengaruhi performa server situs web tersebut, tetapi memengaruhi performa browser dan komputer pengguna. Semakin kompleks kode JavaScript, semakin banyak memori komputer yang dikonsumsi oleh browser.

Pada mata kuliah PBP, kita hanya akan fokus pada *client-side JavaScript*.

Tahapan Eksekusi JavaScript oleh Browser

Perhatikan diagram berikut untuk mengamati tahapan eksekusi JavaScript oleh browser.



Gambar 1: Tahapan eksekusi JavaScript

Saat mengurai (*parsing*) halaman HTML, browser menangani setiap tag `<script>` yang ditemuinya. Jika tag tersebut berisi kode *embedded* JavaScript, browser langsung menjalankan kode itu. Jika tag tersebut merujuk berkas JavaScript eksternal, browser mengunduh berkas itu terlebih dahulu, lalu menjalankannya.

Penulisan JavaScript

Penulisan JavaScript dapat dilakukan dengan *embedded JavaScript* atau *external JavaScript*. Kode JavaScript dapat didefinisikan atau dituliskan secara *embedded* pada berkas HTML maupun secara terpisah pada berkas tersendiri. Jika ditulis dalam berkas terpisah dari HTML, ekstensi berkas yang digunakan untuk berkas JavaScript adalah `.js`. Berikut contoh beberapa pendefinisian dari JavaScript.

JavaScript dapat diletakkan pada *head* atau *body* dari halaman HTML. Kode *embedded* JavaScript **harus** dimasukkan di antara tag `<script>` dan `</script>`. Pengecualiannya adalah *inline event handler* seperti `onclick="..."` yang ditulis langsung sebagai atribut elemen HTML. Kamu dapat meletakkan lebih dari satu *tag script* yang berisi JavaScript pada suatu berkas HTML.

Embedded JavaScript pada HTML

```
<script type="text/JavaScript">
    alert("Hello World!");
</script>
```

External JavaScript pada HTML

```
<script type="text/JavaScript" src="js/script.js"></script>
```

```
alert("Hello World!");
```

Pada berkas JavaScript eksternal, tag `<script>` tidak perlu lagi ditambahkan.

Memisahkan JavaScript pada berkas tersendiri dapat memberikan beberapa keuntungan, seperti kode dapat digunakan di berkas HTML lain, kode JavaScript dan HTML tidak bercampur sehingga kita lebih fokus saat mengembangkan aplikasi, serta proses pemuatan halaman menjadi lebih cepat. Berkas `.js` biasanya di-*cache* oleh browser. Jika kita membuka halaman yang sama dan tidak ada perubahan pada berkas `.js`, browser tidak meminta berkas `.js` tersebut ke server lagi, melainkan menggunakan berkas dari *cache* yang sudah disimpan sebelumnya.

Eksekusi JavaScript

Setelah JavaScript dimuat, browser langsung mengeksekusi kodenya. Jika kode tersebut bukan *event-triggered*, kode langsung dieksekusi. Jika kode dipicu *event*, kode hanya dieksekusi saat *event* tersebut terjadi (*triggered*).

```
// langsung dieksekusi
alert("Hello World");

// langsung dieksekusi
var obj = document.getElementById("object");
// langsung dieksekusi, menambahkan event handler onclick untuk element object
```

```
obj.onclick = function () {  
    // hanya dieksekusi jika element 'object' di klik  
    alert("You just clicked the object!");  
};
```

Sintaks JavaScript

Variabel

Mendefinisikan variabel pada JavaScript cukup mudah. Contohnya seperti berikut.

```
var example = 0; // var example merupakan sebuah bilangan  
var example = "example"; // var example merupakan sebuah string  
var example = true; // var example merupakan sebuah boolean
```

Variabel JavaScript dapat menampung berbagai tipe data, misalnya string, number, boolean, dan objek. Di Java, tipe data ditulis pada deklarasi variabel. Contohnya, jika kamu ingin membuat variabel bertipe `int`, sintaksnya adalah `int x = 9`. Sebaliknya, JavaScript bersifat *dynamically typed*. Kamu tidak perlu menuliskan tipe data pada deklarasi variabel karena tipe sebuah variabel ditentukan oleh nilai yang disimpannya saat *runtime*, seperti pada contoh di atas.

Ada beberapa aturan dalam pemilihan *identifier* atau nama variabel dalam JavaScript. Karakter pertama **harus** berupa alfabet, *underscore* (`_`), atau karakter dolar (`$`). Selain itu, *identifier* JavaScript bersifat *case sensitive*.

Selain `var`, kamu juga akan sering menemui `let` dan `const` pada kode di tutorial ini. `let` digunakan untuk variabel yang dapat diberi nilai baru, sedangkan `const` digunakan untuk variabel yang tidak dapat diberi nilai baru (*reassign*) setelah dideklarasikan.

Penggabungan String

Dalam JavaScript, kita juga dapat menyambungkan `string` dengan `string` lainnya seperti pada Java.

```
var str1 = "PBP" + " " + "Fun";  
var str2 = "PBP";  
var str3 = "Fun";  
var str4 = str2 + " " + str3;  
var str5 = "Fun";  
var str6 = `PBP ${str5}`; // Memiliki hasil yang sama seperti "PBP" + " " + str5
```

Ruang Lingkup JavaScript

Variabel Lokal

Variabel yang didefinisikan **di dalam** fungsi bersifat lokal sehingga hanya dapat diakses oleh kode di dalam fungsi tersebut.

```
// kode di luar fungsi thisFunction() tidak dapat mengakses variabel courseName
function thisFunction() {
    var courseName = "PBP";
    // kode di dalam fungsi ini dapat mengakses variabel courseName
}
```

Variabel Global

Variabel yang didefinisikan **di luar** fungsi bersifat global dan dapat diakses oleh semua *script* pada halaman yang sama, bukan hanya oleh kode dalam berkas JavaScript tersebut.

```
var courseName = "PBP";
function thisFunction() {
    // kode di dalam fungsi ini dapat mengakses variabel courseName
}
```

Variabel Auto Global

Jika sebuah nilai di-*assign* ke variabel yang belum dideklarasikan, variabel tersebut otomatis menjadi variabel global, walaupun *assignment* tersebut berada di dalam suatu fungsi. Perilaku ini hanya berlaku di luar *strict mode*.

```
thisFunction(); // fungsi thisFunction() perlu dipanggil terlebih dahulu
console.log(courseName); // cetak "PBP" pada JavaScript console
function thisFunction() {
    courseName = "PBP";
}
```

Mengakses Variabel Global dari HTML

Kamu dapat mengakses variabel yang berada dalam berkas JavaScript pada berkas HTML yang memuat berkas JavaScript tersebut.

```
...
<input type="text" onclick="this.value=courseName" />
...
```

```
...
var courseName = "PBP";
...
```

Fungsi dan Event

Fungsi adalah sekumpulan kode yang dapat dipanggil dari bagian mana pun dalam program (mirip dengan *method* pada Java). Fungsi mengurangi redundansi kode. Selain itu, fungsi pada

JavaScript memudahkan pemanggilan kode yang sama dari berbagai bagian halaman. Fungsi dapat dipanggil oleh fungsi lain dan dapat juga dipanggil karena *event* (akan dijelaskan di bawah). Sebagai contoh, berikut kode yang terdapat pada `index.html`.

```
...  
<input type="button" value="magicButton" id="magicButton" onclick="hooray();" />  
...
```

Kemudian berikut adalah kode pada `javascript.js`.

```
...  
function hooray() {  
    alert("Yahoo!");  
}  
...
```

Apabila `magicButton` ditekan, atribut `onclick` akan menjalankan fungsi `hooray()` pada `javascript.js`, lalu muncul *alert* sesuai yang sudah di-assign sebelumnya.

Selain penulisan `function` biasa, JavaScript juga mendukung *arrow function*, yaitu penulisan singkat (`parameter`) => { ... } yang akan sering kamu temui pada kode di tutorial ini, misalnya `setTimeout(() => { ... }, 300)`.

`onclick` adalah atribut *event handler* untuk *event click*. *Event* adalah kejadian pada halaman web (misalnya klik atau input) yang dapat ditanggapi oleh JavaScript sehingga situs web menjadi dinamis. Atribut `onclick` menentukan apa yang akan dilakukan JavaScript jika elemen tersebut ditekan. *Event* biasanya dipasangkan dengan sebuah fungsi yang berisi perintah-perintah untuk JavaScript. Selain itu, ada banyak *event* lain, misalnya `onchange`, `onmouseover`, dan `onmouseout`. Daftar lengkapnya bisa kamu baca pada tautan ini.

JavaScript DOM

HTML DOM

HTML DOM (*Document Object Model*) adalah standar bagaimana mengubah, mengambil, dan menghapus elemen HTML. HTML DOM dapat diakses melalui JavaScript atau dengan bahasa pemrograman lainnya. Detail lengkapnya dapat dilihat di sini.

Berikut adalah contoh implementasinya.

```
...  
<div>  
    <p onclick="myFunction()" id="demo">Example of HTML DOM</p>  
</div>  
...
```

```

...
function myFunction() {
    document.getElementById("demo").innerHTML = "YOU CLICKED ME!";
}
...

```

CSS DOM

Sama dengan HTML DOM, CSS DOM dapat mengubah CSS secara dinamis melalui JavaScript. Detail lengkapnya dapat dilihat di sini.

Berikut adalah contoh implementasinya.

```

...
<p id="blueText" onclick="changeColor()">Click me v2</p>
...

```

```

...
function changeColor(){
    document.getElementById("blueText").style.color="blue";
}
...

```

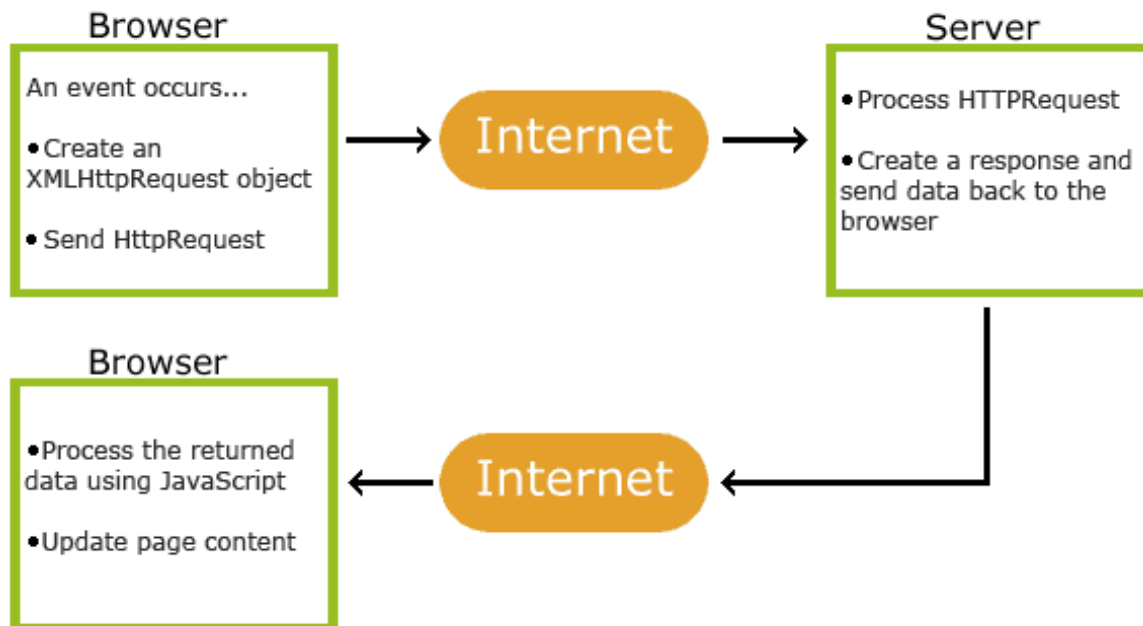
AJAX

Pengenalan AJAX

AJAX merupakan singkatan dari *Asynchronous JavaScript And XML*.

AJAX bukan bahasa pemrograman, melainkan teknik yang memadukan objek permintaan bawaan browser (`XMLHttpRequest` atau `fetch()`, untuk meminta data dari server) dengan JavaScript dan HTML DOM (untuk menampilkan data). Selain XML, AJAX juga dapat mengirim data berupa teks atau JSON. AJAX memungkinkan halaman web memperbarui data secara asinkron dengan bertukar data dengan server di balik layar. Hal tersebut berarti bahwa kita dapat memperbarui sebagian elemen data pada halaman tanpa harus me-*reload* halaman secara keseluruhan.

Berikut ini adalah diagram cara kerja AJAX.



Gambar 2: Cara kerja AJAX. Sumber: w3schools

1. Sebuah *event* terjadi pada halaman web (contohnya tombol *submit data* ditekan)
2. Sebuah objek `XMLHttpRequest` dibuat oleh JavaScript
3. Objek `XMLHttpRequest` mengirimkan permintaan ke server
4. Server memproses permintaan tersebut
5. Server mengembalikan respons kepada halaman web
6. Respons dibaca oleh JavaScript
7. Aksi berikutnya akan dipicu oleh JavaScript sesuai dengan langkah yang dibuat (contohnya memperbarui data di halaman tersebut)

`XMLHttpRequest` sebelumnya merupakan cara standar untuk melakukan permintaan AJAX di JavaScript. Namun, `XMLHttpRequest` memiliki beberapa kelemahan, seperti penanganan yang kurang rapi ketika bekerja dengan `Promise` dan `callback` serta keterbatasan dalam mendukung alur kode yang lebih modern.

Oleh karena itu, `fetch()` diperkenalkan sebagai API baru untuk melakukan permintaan HTTP dengan sintaks yang lebih sederhana dan mendukung `Promise` secara langsung. Hal ini memungkinkan pengembang menulis kode yang lebih mudah dibaca, dikelola, dan lebih cocok dengan paradigma asinkron modern seperti `async/await`. `fetch()` juga lebih fleksibel dalam menangani format data yang berbeda. Namun, `fetch()` tidak menolak (*reject*) `Promise` ketika server membalas dengan status HTTP 4xx atau 5xx. Oleh karena itu, kode harus memeriksa `response.ok`, seperti yang dilakukan kode pada tutorial ini. Penjelasan lebih lanjut mengenai perbedaan `fetch()` dan `XMLHttpRequest` dapat dilihat di tautan ini.

Pada PBP kali ini, kamu akan melakukan AJAX pada browser menggunakan fungsi `fetch()` yang disediakan oleh browser sebagai bagian dari Web API, bukan oleh bahasa JavaScript itu sendiri.

Sebagai gambaran besar, penggunaan `fetch()` untuk melakukan pemanggilan AJAX dapat dilihat di tautan ini.

Fetch API

Fetch API merupakan API yang menyediakan antarmuka JavaScript untuk mengakses dan memanipulasi sumber daya (termasuk di seluruh jaringan) menggunakan `Promise`. Fetch API distandarisasi oleh WHATWG sebagai bagian dari *Fetch Standard*, dan telah didukung oleh browser modern sejak tahun 2015. API ini merupakan pengganti yang lebih kuat dan fleksibel untuk `XMLHttpRequest`. Fetch API secara umum digunakan untuk mengimplementasikan AJAX secara lebih mudah daripada AJAX dengan `XMLHttpRequest`.

Fungsi `fetch()` memiliki dua parameter, yaitu:

- `resource` berisi URL dari sumber daya yang akan diminta.
- `options` (opsional) adalah objek pengaturan permintaan, seperti `method` (metode HTTP yang akan digunakan), `headers` (header HTTP yang akan dikirim), dan `body` (isi dari permintaan HTTP).

Fungsi `fetch()` mengembalikan `Promise` yang menghasilkan objek `Response` setelah server membalas. Objek `Response` memiliki beberapa properti, yaitu:

- `ok` bernilai `true` jika kode `status` HTTP berada pada rentang 200 sampai dengan 299.
- `status` berisi kode status HTTP dari respons.
- `headers` berisi header HTTP dari respons.
- `body` berisi isi dari respons.

Kamu dapat mempelajari Fetch API lebih lanjut pada tautan ini.

Keyword Async dan Await

Sebelum mempelajari penggunaan fungsi `fetch()`, ada baiknya kita mempelajari *keyword* `async` dan `await`. Kedua *keyword* ini membuat kode asinkron (seperti pemanggilan `fetch()`) lebih mudah dibaca tanpa *callback* yang bersarang.

`async` dan `await` merupakan *keyword* baru yang diperkenalkan di ECMAScript 2017. `async` adalah *keyword* yang diletakkan sebelum deklarasi fungsi untuk menandai bahwa fungsi tersebut adalah *async function*, yaitu fungsi yang selalu mengembalikan `Promise`. Sementara itu, `await` adalah *keyword* yang hanya bisa digunakan di dalam *async function* dan berfungsi untuk “menunggu” `Promise` selesai diproses sebelum melanjutkan ke baris kode berikutnya. Tanpa `await`, sebuah `Promise` akan tetap berjalan di belakang layar dan kode berikutnya akan langsung dieksekusi tanpa menunggu hasilnya.

Sebagai contoh, perhatikan potongan kode berikut:

```
async function ambilData() {  
  const hasil = await fetch('https://contoh-api.com/data');  
  console.log(hasil);  
}
```

Pada kode di atas, *keyword* `async` yang diletakkan sebelum `function` menandai bahwa `ambilData` adalah *async function*. Selanjutnya, *keyword* `await` yang diletakkan sebelum `fetch()` membuat eksekusi kode menunggu hingga proses `fetch()` selesai sebelum baris `console.log(hasil)` dijalankan.

Kamu dapat mempelajari *keyword* `async` dan `await` lebih lanjut pada tautan ini.

Penggunaan Fetch API

Fetch API menyediakan fungsi `fetch()` global untuk mengambil sumber daya secara asinkron melalui jaringan.

Tidak seperti `XMLHttpRequest` yang merupakan API berbasis *callback*, Fetch API berbasis `Promise` dan menyediakan alternatif yang lebih baik dan dapat dengan mudah digunakan pada *service worker*. Fetch API juga mengintegrasikan konsep HTTP tingkat lanjut seperti CORS dan ekstensi lain ke HTTP.

Berikut adalah contoh penggunaan Fetch API dengan *keyword* `async` dan `await` untuk melakukan AJAX.

```
async function fetchData() {
    const response = await fetch("https://jokes-bapack2-api.yuana.id/v1/text/random");
    const data = await response.json();
    return data;
}

async function main() {
    const joke = await fetchData();
    console.log(joke);
}

main();
```

Kode di atas akan melakukan AJAX untuk meminta data dari API lelucon bapak-bapak masa kini secara asinkron. Karena `await` hanya bisa digunakan di dalam *async function*, kita perlu membungkus pemanggilan `fetchData()` di dalam fungsi `main()` yang juga ditandai sebagai *async function*. Hasil dari AJAX akan disimpan dalam variabel `joke`.

Kamu dapat mempelajari penggunaan Fetch API lebih lanjut pada tautan ini.

Pre-Tutorial Notes

Sebelum melanjutkan Tutorial 05 ini, kami mengharapkan kamu memastikan hal-hal berikut:

- Jika kamu tidak sengaja mem-*push* berkas sensitif seperti `.env`, `.env.prod`, `db.sqlite3`, atau folder *virtual environment*, hentikan pelacakan Git dengan perintah berikut:

```
git rm --cached --ignore-unmatch .env .env.prod db.sqlite3
git rm -r --cached --ignore-unmatch env/ .venv/
```

Kedua perintah di atas menghentikan pelacakan Git tanpa menghapus salinan lokal. Opsi `--ignore-unmatch` melewati *path* yang tidak dilacak Git. Kedua perintah ini tidak menghapus berkas dari riwayat *commit* sebelumnya. Jika *credential* pernah ter-*push*, ganti *credential* tersebut dengan yang baru karena menghentikan pelacakan tidak menghapusnya dari riwayat Git.

Periksa apakah `.gitignore` sudah berisi berkas sensitif tersebut:

```
cat .gitignore
```

Jika belum ada, tambahkan ke `.gitignore`:

```
.env*
db.sqlite3
env/
.venv/
```

Kemudian buat *commit* untuk pembersihan tersebut:

```
git add .
git commit -m "cleanup"
```

Dengan demikian, kita dapat meminimalkan risiko keamanan dari *credential* yang terekspos di repositori publik.

- **Struktur direktori `myportfolio` setelah Tutorial 04**

```
myportfolio
├─ env/                # atau .venv/, sesuai virtual environment kamu
├─ .git/
├─ .gitignore
├─ requirements.txt
├─ db.sqlite3
├─ manage.py
├─ main
│   ├─ migrations
│   ├─ __init__.py
│   ├─ admin.py
│   ├─ apps.py
│   ├─ forms.py
│   └─ models.py
```

```

|   ├── tests.py
|   ├── urls.py
|   └── views.py
├── portofolio          # folder konfigurasi utama proyek
|   ├── __init__.py
|   ├── asgi.py
|   ├── settings.py    # semua konfigurasi proyek ada di sini
|   ├── urls.py        # daftar routing URL
|   ├── views.py
|   └── wsgi.py
├── static
|   ├── css
|   │   └── style.css
|   ├── img
|   │   └── burhan.png
└── templates
    ├── base.html
    ├── index.html
    ├── experience.html
    ├── login.html
    ├── project.html
    ├── projects_form.html
    ├── register.html
    └── components
        ├── project_delete_modal.html
        └── project_star.html

```

Implementasi Notifikasi Toast

Toast merupakan elemen antarmuka pengguna yang dirancang untuk menampilkan pesan singkat atau notifikasi secara nonintrusif. Dalam tutorial ini, kita akan mengembangkan komponen toast yang dapat digunakan kembali (*reusable*) dan berfungsi untuk memberikan umpan balik visual, misalnya saat sebuah proyek berhasil ditambahkan.

Langkah 1: Konstruksi Komponen Utama Toast

Buka folder `templates/components` pada direktori *root* proyek kamu. Buatlah sebuah berkas baru bernama `toast.html`, lalu sisipkan kode berikut:

```

{% load static %}
<div id="toast-component" class="toast-component toast-hidden"
popover="manual">
  <div class="toast-content">
    <h3 id="toast-title" class="toast-title">This is a Sample Title</h3>
    <p id="toast-message" class="toast-message">This is a Sample Message</p>
  </div>
</div>

```

```

    </div>
</div>
<script src="{% static 'js/toast.js' %}"></script>

```

Atribut `popover="manual"` membuat toast dapat ditampilkan lewat JavaScript ke *top layer* browser sehingga selalu berada di atas elemen lain, termasuk modal.

Langkah 2: Tambahkan CSS untuk Toast

Untuk mempercantik tampilan toast, tambahkan baris *styling* berikut ke bagian paling bawah dari berkas `static/css/style.css` milikmu:

```

/* toast */
.toast-component {
    position: fixed;
    inset: auto 2rem 2rem auto;
    margin: 0;
    padding: 1rem 1.5rem;
    border-radius: var(--radius);
    box-shadow: 0 10px 25px rgba(28, 25, 23, 0.15);
    display: none;
    align-items: center;
    gap: 1rem;
    transition: all 0.3s ease;
    background-color: var(--paper);
    border: 1px solid var(--line);
    color: inherit;
}

.toast-component:popover-open {
    display: flex;
}

.toast-hidden {
    opacity: 0;
    transform: translateY(150%);
    pointer-events: none;
}

.toast-show {
    opacity: 1;
    transform: translateY(0);
}

```

```

.toast-title {
  font-size: 1rem;
  font-weight: 700;
  margin-bottom: 0.15rem;
  font-family: "Space Grotesk", -apple-system, sans-serif;
}

.toast-message {
  font-size: 0.85rem;
  color: var(--text-muted);
}

/* Modifikasi border kiri berdasarkan tipe notifikasi */
.toast-success { border-left: 4px solid #22c55e; }
.toast-error { border-left: 4px solid #b42318; }
.toast-normal { border-left: 4px solid var(--accent); }

```

Langkah 3: Implementasi Logika JavaScript untuk Toast

Untuk mengimplementasikan fungsionalitas utama dari toast, buatlah sebuah berkas bernama `toast.js` di dalam folder `static/js`. Kemudian, tambahkan kode berikut ke dalam berkas tersebut:

```

let toastTimer;

function showToast(title, message, type = 'normal', duration = 3000) {
  const toastComponent = document.getElementById('toast-component');
  const toastTitle = document.getElementById('toast-title');
  const toastMessage = document.getElementById('toast-message');

  if (!toastComponent) return;

  // Hapus class tipe sebelumnya
  toastComponent.classList.remove('toast-success', 'toast-error', 'toast-normal');

  // Terapkan class baru berdasarkan tipe
  if (type === 'success') {
    toastComponent.classList.add('toast-success');
  } else if (type === 'error') {
    toastComponent.classList.add('toast-error');
  } else {
    toastComponent.classList.add('toast-normal');
  }
}

```

```

// Perbarui konten teks
toastTitle.textContent = title;
toastMessage.textContent = message;

// Batalkan timer sebelumnya jika toast masih tampil
clearTimeout(toastTimer);

// Animasi muncul
if (!toastComponent.matches(':popover-open')) {
    toastComponent.showPopover();
    void toastComponent.offsetHeight; // paksa browser menghitung style agar
transisi berjalan
}
toastComponent.classList.remove('toast-hidden');
toastComponent.classList.add('toast-show');

// Animasi hilang otomatis
toastTimer = setTimeout(() => {
    toastComponent.classList.remove('toast-show');
    toastComponent.classList.add('toast-hidden');
    toastTimer = setTimeout(() => toastComponent.hidePopover(), 300);
}, duration);
}

```

Penjelasan fungsi `showToast`:

Fungsi `showToast` memiliki empat parameter:

- `title` adalah judul notifikasi yang akan ditampilkan.
- `message` adalah pesan utama yang akan ditampilkan.
- `type` adalah tipe notifikasi (seperti `'success'`, `'error'` atau `'normal'`). Parameter ini menentukan skema warna notifikasi.
- `duration` adalah durasi notifikasi muncul di layar (dalam milidetik).

Fungsi ini secara dinamis mengubah kelas CSS dan konten elemen HTML berdasarkan parameter yang diberikan. Secara rinci, `showToast` melakukan langkah-langkah berikut:

- Fungsi ini mengakses elemen-elemen HTML toast melalui ID-nya, yaitu `toast-component`, `toast-title`, dan `toast-message`.
- Fungsi ini menghapus kelas-kelas tipe sebelumnya untuk memastikan notifikasi memiliki gaya yang benar sebelum menerapkan gaya yang baru.
- Fungsi ini menambahkan kelas CSS yang sesuai (`toast-success`, `toast-error` atau `toast-normal`) untuk mengubah warna *border* berdasarkan parameter `type`.
- Fungsi ini memperbarui teks judul dan pesan notifikasi.

- Fungsi ini menampilkan toast dengan `showPopover()` agar toast berada di *top layer* browser, di atas elemen lain termasuk modal. Setelah itu, fungsi ini menghapus kelas `toast-hidden` dan menambahkan kelas `toast-show` sehingga toast tampak muncul dari bawah layar.
- Fungsi ini menggunakan `setTimeout` untuk menyembunyikan toast secara otomatis setelah durasi yang ditentukan, lalu memanggil `hidePopover()` setelah animasinya selesai. `clearTimeout` membatalkan *timer* sebelumnya agar toast yang dipanggil berturut-turut tidak disembunyikan terlalu cepat.

Langkah 4: Integrasi Komponen Toast ke dalam Base Template

Setelah komponen toast selesai dibuat, kita perlu menyisipkan komponen tersebut ke dalam berkas `base.html` agar fungsi `showToast` dapat diakses dari semua halaman yang mewarisi atau meng-*extend* *base template* tersebut. Sisipkan `{% include 'components/toast.html' %}` tepat sebelum tag penutup `</body>`:

```
<!-- Kode footer kamu sebelumnya -->
<footer class="site-footer">
  <div class="container">
    <p>&copy; 2026 {{ name }}. Fakultas Ilmu Komputer, Universitas
Indonesia.</p>
  </div>
</footer>

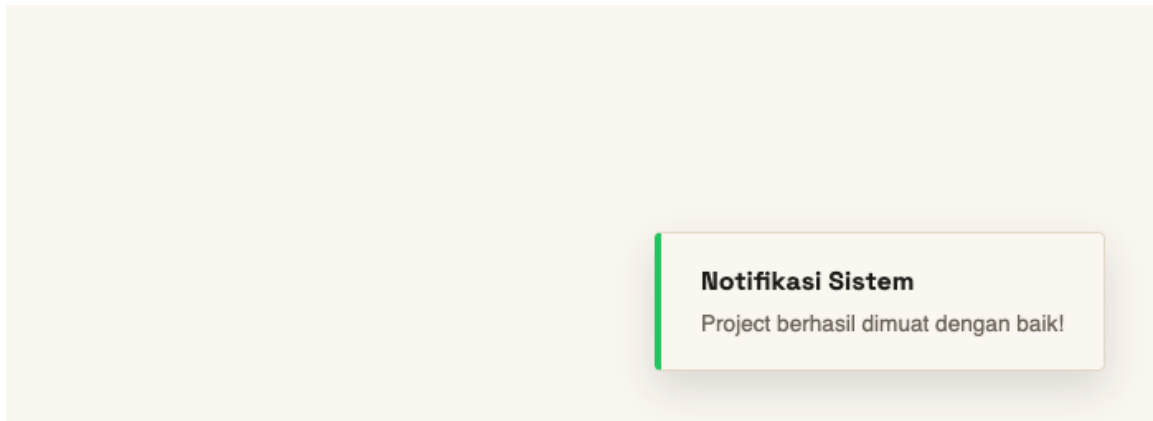
{% include 'components/toast.html' %}
</body>
</html>
```

Langkah 5: Pengujian Fungsionalitas Toast

Untuk memverifikasi apakah notifikasi toast sudah berfungsi dengan baik, kita dapat menambahkan tombol uji coba ke halaman mana pun yang mewarisi `base.html` (misalnya di `project.html`, tepat di bawah `div project-header`):

```
<button class="button button-secondary" onclick="showToast('Notifikasi Sistem',
'Project berhasil dimuat dengan baik!', 'success')">
  Test Toast Notifikasi
</button>
```

Saat tombol tersebut diklik, sebuah notifikasi toast akan muncul di layar dengan animasi dari bawah dan menghilang dengan sendirinya setelah 3 detik.



Gambar 3: Tampilan toast setelah menekan tombol uji coba

Nantinya, kamu bisa memanggil fungsi `showToast()` ini dalam penanganan respons AJAX saat membuat atau menghapus proyek agar pengguna mendapat konfirmasi yang tidak mengganggu.

Menampilkan Data Proyek dengan AJAX

! Catatan Penting

Kode pada tutorial ini hanya contoh. Sesuaikan bagian, objek, dan *field* dengan proyekmu. Jangan melakukan *copy-paste* secara langsung tanpa penyesuaian jika implementasi yang kamu buat berbeda!

Kita akan mengubah cara halaman Projects menampilkan data. Sebelumnya, seluruh data halaman sudah dimuat dari server sejak permintaan pertama. Kini, kita akan mengimplementasikan AJAX sehingga browser hanya memuat kerangka halamannya terlebih dahulu, lalu mengambil data proyek secara terpisah.

Langkah 1: Memperbarui Fungsi `get_projects_json`

Agar fitur *star* tetap berfungsi saat data diubah ke JSON, kita tidak bisa lagi menggunakan `serializers.serialize` bawaan Django karena `serializers.serialize` tidak dapat memeriksa pengguna yang sedang *login*. Kita harus merakit JSON-nya secara manual.

Buka `main/views.py` dan perbarui fungsi `get_projects_json` menjadi seperti ini:

```
from django.http import JsonResponse

def get_projects_json(request):
    title_query = request.GET.get("title", "").strip()
    projects = Project.objects.prefetch_related('starred_by').all()

    if title_query:
```

```

        projects = projects.filter(title__icontains=title_query)

# Konstruksi data JSON secara manual agar bisa menyisipkan logika Star
data = []
for project in projects:
    starred_users = project.starred_by.all()
    is_starred = request.user in starred_users if request.user.is_authenticated
else False
    starred_by_names = ", ".join([u.username for u in starred_users])

    data.append({
        "pk": str(project.id),
        "fields": {
            "title": project.title,
            "description": project.description,
            "tech_stack": project.tech_stack,
            "project_url": project.project_url,
            "project_image_url": project.project_image_url,
            "star_count": starred_users.count(),
            "is_starred": is_starred,
            "starred_by_names": starred_by_names,
        }
    })

return JsonResponse(data, safe=False)

```

Kode di atas menggunakan perulangan untuk mengubah setiap objek `Project` menjadi sebuah *dictionary*. Di dalam perulangan tersebut, kita juga memeriksa apakah `request.user` ada di dalam daftar akun yang memberi *star* pada proyek tersebut (`is_starred`) dan menghitung total *star*. `JsonResponse` kemudian mengirimkan data utuh ini ke browser.

Langkah 2: Menyederhanakan Fungsi `show_projects`

Karena data sekarang diambil mandiri oleh JavaScript, *view* yang bertugas menampilkan halaman HTML tidak perlu lagi mengirimkan variabel `project_list`.

Perbarui fungsi `show_projects` di `main/views.py`:

```

def show_projects(request):
    title_query = request.GET.get("title", "").strip()

    context = {
        "name": "Burhan",
        "title_query": title_query,
    }

```

```

}
return render(request, "project.html", context)

```

Langkah 3: Menyiapkan Template `project.html` untuk AJAX

Buka berkas `project.html`. Kita akan mengganti perulangan *template* Django (`{% for project in project_list %}`) dengan wadah kosong yang akan diisi oleh JavaScript. Ubah `<form>` pencarian dan bagian di bawahnya menjadi seperti ini:

```

...
<form id="project-search-form" class="project-search">
  <input type="search"
    id="search-input"
    name="title"
    value="{{ title_query }}"
    placeholder="Cari berdasarkan nama proyek"
    class="project-search__input"
    aria-label="Cari berdasarkan nama proyek">
  <button type="submit" class="button">Cari</button>
</form>

<!-- Loading State -->
<div id="loading" class="empty-state hide">
  <p>Memuat projects...</p>
</div>

<!-- Error State -->
<div id="error" class="empty-state hide">
  <p>Gagal memuat data projects. Silakan coba lagi.</p>
</div>

<!-- Empty State -->
<div id="empty" class="empty-state hide">
  <p>Belum ada proyek yang ditambahkan atau ditemukan.</p>
</div>

<!-- Project Grid -->
<div id="grid" class="experience-grid project-grid hide"></div>
...

```

i Catatan

Form pencarian kini diberi atribut `id`, dan kita menghapus atribut `method` dan `action` bawaannya karena peran kedua atribut tersebut akan digantikan penuh oleh JavaScript. Atribut `value` tetap dipertahankan agar kata kunci dari alamat `?title=` tetap terisi.

Perhatikan juga bahwa kita menggunakan kelas `hide` untuk menyembunyikan elemen `loading`, `error`, dan `empty` secara *default*. Jika kamu belum menambahkan kelas ini pada tutorial-tutorial sebelumnya, pastikan `style.css` kamu sudah memiliki aturan berikut:

```
.hide {  
  display: none !important;  
}
```

Langkah 4: Menambahkan Script AJAX Inline

Terakhir, tambahkan logika JavaScript untuk mengambil JSON dan merakitnya menjadi elemen HTML. Tambahkan blok `<script>` ini tepat sebelum tag `{% endblock content %}` di `project.html`:

```
<script>  
// Konfigurasi  
const BASE_PROJECTS_ENDPOINT = "{% url 'main:get_projects_json' %}";  
const IS_SUPERUSER = "{ { user.is_superuser|yesno:'true,false' }}" === "true";  
let projectsAbortController;  
  
// Elemen DOM  
const loadingState = document.getElementById('loading');  
const errorState = document.getElementById('error');  
const emptyState = document.getElementById('empty');  
const gridContainer = document.getElementById('grid');  
const searchForm = document.getElementById('project-search-form');  
const searchInput = document.getElementById('search-input');  
  
// Menyembunyikan/Menampilkan section halaman  
function displayPageSection({ showLoading = false, showError = false,  
showEmpty = false, showGrid = false }) {  
  loadingState.classList.toggle('hide', !showLoading);  
  errorState.classList.toggle('hide', !showError);  
  emptyState.classList.toggle('hide', !showEmpty);  
  gridContainer.classList.toggle('hide', !showGrid);  
}
```

```

// Membuat elemen card project
function buildProjectCardElement(item) {
    const project = item.fields;
    const projectId = item.pk;

    const articleElement = document.createElement('article');
    articleElement.className = 'experience-card';

    const imageHtml = project.project_image_url
        ? ``
        : '';

    const urlHtml = project.project_url
        ? `<a href="${project.project_url}" class="button">Lihat Project</a>`
        : '';

    // Dummy UUID digunakan untuk digantikan oleh ID asli dari data JSON
    const deleteUrl = "{% url 'main:delete_project'
    '00000000-0000-0000-0000-000000000000' %}"
        .replace('00000000-0000-0000-0000-000000000000', projectId);
    const starUrl = "{% url 'main:toggle_star'
    '00000000-0000-0000-0000-000000000000' %}"
        .replace('00000000-0000-0000-0000-000000000000', projectId);

    const deleteHtml = IS_SUPERUSER
        ? `<form method="post" action="${deleteUrl}" style="display:inline;">
        {% csrf_token %}
        <button type="submit" class="button button-danger" onclick="return
        confirm('Yakin ingin menghapus project ini?');">Hapus</button>
        </form>`
        : '';

    const isStarredClass = project.is_starred ? " is-starred" : "";
    const starText = project.is_starred ? "Unstar" : "Star";
    const starTitle = project.star_count > 0
        ? `Dibintangi oleh ${project.starred_by_names}`
        : "Jadilah yang pertama memberi star";

    // Komponen card
    const completeCardHtml = `
        ${imageHtml}

```

```

    <h2>${project.title}</h2>
    <span class="experience-category">${project.tech_stack}</span>
    <p class="experience-description">${project.description}</p>
    <div class="project-card-actions">
      <div class="project-actions">
        ${urlHtml}

        <form method="post" action="${starUrl}" class="star-form">
          {% csrf_token %}
          <button type="submit"
            class="button button-star${isStarredClass}"
            title="${starTitle}">
            <span aria-hidden="true">★</span>
            ${starText}
            <span class="star-count">${project.star_count}</span>
          </button>
        </form>

        ${deleteHtml}
      </div>
    </div>
  `;

  articleElement.innerHTML = completeCardHtml;
  return articleElement;
}

// Fetch data project
async function fetchProjects(searchQuery = "") {
  if (projectsAbortController) projectsAbortController.abort();
  projectsAbortController = new AbortController();

  try {
    displayPageSection({ showLoading: true });

    const url = searchQuery ? `${BASE_PROJECTS_ENDPOINT}?
title=${encodeURIComponent(searchQuery)}` : BASE_PROJECTS_ENDPOINT;

    const response = await fetch(url, {
      headers: { 'Accept': 'application/json' },
      signal: projectsAbortController.signal,
    });

```

```

    if (!response.ok) throw new Error('Failed to fetch data');

    const projectData = await response.json();

    if (projectData.length === 0) {
        displayPageSection({ showEmpty: true });
    } else {
        gridContainer.innerHTML = '';
        projectData.forEach(item => {
            gridContainer.appendChild(buildProjectCardElement(item));
        });
        displayPageSection({ showGrid: true });
    }
} catch (error) {
    if (error.name === 'AbortError') return;
    console.error('Error loading projects:', error);
    displayPageSection({ showError: true });
}

// Event Handlers untuk Form Search
searchForm.addEventListener('submit', function(e) {
    e.preventDefault(); // Mencegah reload halaman
    fetchProjects(searchInput.value.trim()); // Jalankan pencarian via AJAX
});

// Start application
fetchProjects(searchInput.value.trim());
</script>

```

Penjelasan Logika JavaScript

- `displayPageSection` menampilkan bagian halaman yang relevan (seperti teks *loading*, *error* atau hasil pencarian) dengan memanfaatkan kelas `hide`. Fungsi ini menerima satu objek dengan nilai *default false* untuk setiap bagian sehingga kita cukup menuliskan bagian yang ingin ditampilkan, misalnya `displayPageSection({ showGrid: true })`.
- `buildProjectCardElement` menerima satu objek JSON proyek dan merakitnya menjadi elemen `<article>` lengkap.
- `fetchProjects` memanggil Fetch API untuk meminta data ke `/api/projects/`. Setelah data JSON diterima, fungsi ini membersihkan wadah HTML lama dan me-*render* hasil pencarian baru secara dinamis. `AbortController` membatalkan permintaan sebelumnya yang belum selesai sehingga hasil pencarian lama tidak menimpa hasil yang terbaru.

- Penanganan *event* pencarian memanggil `e.preventDefault()` untuk memastikan halaman tidak di-*reload* saat pengguna menekan tombol **Cari**. Sebagai gantinya, fungsi `fetchProjects` dipanggil dengan parameter kata kunci yang diketikkan pengguna.

Menerapkan Search Debouncing pada Pencarian Proyek

Pada langkah sebelumnya, pencarian hanya dijalankan ketika tombol **Cari** ditekan atau pengguna menekan Enter. Kita dapat membuat pencarian terasa lebih responsif dengan menjalankannya saat pengguna mengetik. Namun, memasang fungsi `fetchProjects` langsung pada *event input* akan membuat browser mengirim satu permintaan untuk setiap karakter. Mencari Django, misalnya, berarti enam permintaan berturut-turut.

Debouncing adalah teknik untuk menunda sebuah fungsi hingga suatu jeda waktu berlalu tanpa *event* baru. Selama pengguna masih mengetik, *timer* sebelumnya dibatalkan dan dimulai lagi. Dengan demikian, browser hanya mengirim permintaan setelah pengguna berhenti mengetik selama sejenak.

Langkah 1: Tambahkan Timer Debounce

Di dalam blok `<script>` pada `project.html`, tambahkan dua baris berikut setelah deklarasi `searchInput`:

```
const SEARCH_DEBOUNCE_DELAY = 300;
let searchDebounceTimer;
```

Nilai 300 berarti browser menunggu 300 milidetik setelah input terakhir. Nilai ini cukup singkat agar hasil pencarian tetap terasa langsung, tetapi cukup panjang untuk menghindari permintaan berulang ketika pengguna sedang mengetik.

Langkah 2: Jalankan Pencarian Saat Pengguna Berhenti Mengetik

Di bagian akhir skrip, ganti *event handler* `searchForm` yang sebelumnya dengan kode berikut:

```
function searchProjects() {
    fetchProjects(searchInput.value.trim());
}

searchInput.addEventListener("input", function() {
    clearTimeout(searchDebounceTimer);

    searchDebounceTimer = setTimeout(function() {
        searchProjects();
    }, SEARCH_DEBOUNCE_DELAY);
});

searchForm.addEventListener("submit", function(event) {
    event.preventDefault();
```

```
clearTimeout(searchDebounceTimer);
searchProjects();
});
```

Penjelasan Kode:

- `clearTimeout(searchDebounceTimer)` membatalkan *timer* dari karakter sebelumnya. Saat pengguna mengetik huruf berikutnya, waktu tunggu selalu dihitung ulang dari awal.
- `setTimeout()` menyimpan fungsi pencarian untuk dijalankan setelah `SEARCH_DEBOUNCE_DELAY` milidetik. Fungsi pencarian tersebut hanya benar-benar berjalan apabila tidak ada input baru sebelum waktunya habis.
- Fungsi `searchProjects()` digunakan oleh dua *event handler* agar logika pencarian tidak ditulis dua kali.
- *Event* `submit` tetap dipertahankan. Tombol **Cari** dan tombol Enter terlebih dahulu membatalkan *timer* yang mungkin masih menunggu, lalu langsung menjalankan pencarian.

Coba ketikkan sebagian judul proyek pada kolom pencarian. Kartu proyek akan diperbarui setelah kamu berhenti mengetik sebentar. Untuk melihat perbedaannya, buka tab **Network** pada DevTools, lalu bandingkan jumlah permintaan sebelum dan sesudah *debouncing* diterapkan.

i Catatan

Debouncing berbeda dari *throttling*. *Debouncing* menunggu hingga *event* berhenti terjadi, sedangkan *throttling* membatasi fungsi agar hanya dapat berjalan sekali dalam interval tertentu. Untuk pencarian yang harus menampilkan hasil setelah pengguna selesai mengetik, *debouncing* lebih sesuai.

Membuat Modal sebagai Form untuk Menambahkan Proyek

Sebelumnya, tombol **Tambah Proyek** membuka halaman form tersendiri. Sekarang kita akan menempatkan form tersebut dalam modal pada halaman Projects. Modal ini menggunakan Popover API, pola yang sama seperti modal konfirmasi hapus proyek pada Tutorial 03. Setelah halaman Projects di-*render* lewat AJAX, tombol hapus kini dibuat oleh JavaScript di dalam kartu proyek, dan tutorial ini memilih `confirm()` bawaan browser yang lebih sederhana untuk konfirmasi hapus di sana (lihat `buildProjectCardElement`). Pola Popover API tetap kita gunakan di sini untuk modal tambah proyek.

Pada bagian berikutnya, form yang sama akan dikirim dengan AJAX. Untuk sekarang, kita menyiapkan struktur form, modal, dan *styling*-nya terlebih dahulu.

Langkah 1: Sediakan `ProjectForm` pada Halaman Projects

Buka `main/views.py`. Pastikan `ProjectForm` telah diimpor, lalu tambahkan form kosong ke `context` pada fungsi `show_projects`:

```

from main.forms import ProjectForm

def show_projects(request):
    title_query = request.GET.get("title", "").strip()

    context = {
        "name": "Burhan",
        "title_query": title_query,
        "form": ProjectForm(),
    }
    return render(request, "project.html", context)

```

`ProjectForm()` tidak menyimpan data apa pun. Objek ini hanya menyediakan *field*, label, *placeholder*, dan validasi yang sudah kamu definisikan pada `main/forms.py` agar bisa di-*render* di dalam modal.

Langkah 2: Ubah Tombol Tambah Proyek

Di `project.html`, ganti tautan **Tambah Proyek** di dalam `project-header` dengan tombol berikut. Tetap letakkan tombol ini di dalam blok `{% if user.is_superuser %}` yang dibuat pada Tutorial 04.

```

<button type="button"
        class="button project-add-button"
        popovertarget="add-project-modal">
    <span aria-hidden="true">+</span>
    Tambah Proyek
</button>

```

Atribut `popovertarget` menghubungkan tombol dengan elemen modal yang memiliki `id` `add-project-modal`. Browser akan menampilkan popover tersebut saat tombol diklik tanpa JavaScript tambahan.

Langkah 3: Tambahkan Komponen Modal dan Form

Buat berkas `templates/components/project_form_modal.html`, lalu isi dengan kode berikut:

```

<div id="add-project-modal"
    class="project-form-modal"
    popover="auto"
    role="dialog"
    aria-modal="true"
    aria-labelledby="add-project-modal-title">
    <button type="button"
        class="project-form-modal__backdrop"

```

```

        popovertarget="add-project-modal"
        popovertargetaction="hide"
        aria-label="Tutup form tambah proyek"></button>

<div class="project-form-modal__content">
    <button type="button"
        class="project-form-modal__close"
        popovertarget="add-project-modal"
        popovertargetaction="hide"
        aria-label="Tutup form tambah proyek">x</button>

    <h2 id="add-project-modal-title">Tambah Proyek Baru</h2>
    <form id="project-form"
        method="post"
        action="{% url 'main:create_project' %}"
        class="project-form">
        {% csrf_token %}
        {% for field in form %}
            <div class="form-group">
                <label for="{{ field.id_for_label }}">{{ field.label }}
</label>

                {{ field }}
                {% for error in field.errors %}
                    <p class="form-error">{{ error }}</p>
                {% endfor %}
            </div>
        {% endfor %}
        <div class="project-form-modal__actions">
            <button type="button"
                class="button button-secondary"
                popovertarget="add-project-modal"
                popovertargetaction="hide">Batal</button>

            <button type="submit" class="button">Tambah Proyek</button>
        </div>
    </form>
</div>
</div>

```

Kemudian, di `project.html`, tambahkan *include* berikut tepat setelah tag penutup `</main>` dan sebelum blok `<script>` di bagian bawah halaman. Bungkus dengan kondisi yang sama agar modal hanya tersedia untuk pemilik portofolio.

```
{% if user.is_superuser %}
    {% include "components/project_form_modal.html" %}
{% endif %}
```

Tombol latar belakang, ikon tutup, dan tombol **Batal** semuanya menggunakan `popovertargetaction="hide"`. Dengan demikian, ketiga tombol tersebut menutup modal yang sama tanpa perlu menulis tiga *event listener* JavaScript terpisah.

Tambahkan juga fungsi berikut di dalam blok `<script>` yang sama di `project.html`, misalnya sebelum pemanggilan `fetchProjects()` di bagian akhir skrip.

```
function closeProjectModal() {
    document.getElementById("add-project-modal").hidePopover();
}
```

Fungsi ini menggunakan *method* Popover API yang sama dengan modal kita. Pada bagian selanjutnya, fungsi `closeProjectModal()` dipanggil setelah server berhasil menyimpan proyek lewat AJAX sehingga modal otomatis tertutup tanpa *me-reload* halaman.

Langkah 4: Tambahkan Style untuk Modal Form

Tambahkan *style* berikut ke `static/css/style.css` setelah *style* `project-delete-modal`. Kelas `form-group`, `form-error`, dan `project-form` yang dibuat pada Tutorial 03 tetap digunakan oleh form ini.

```
.project-form-modal {
    display: none;
    position: fixed;
    inset: 0;
    width: 100%;
    height: 100%;
    max-width: none;
    max-height: none;
    margin: 0;
    padding: 1rem;
    border: 0;
    background: transparent;
    z-index: 10;
    align-items: center;
    justify-content: center;
}

.project-form-modal:popover-open {
    display: flex;
```

```

}

.project-form-modal__backdrop {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  border: 0;
  padding: 0;
  background: rgba(28, 25, 23, 0.58);
  cursor: default;
}

.project-form-modal__content {
  position: relative;
  z-index: 1;
  width: min(100%, 640px);
  max-height: calc(100vh - 2rem);
  overflow-y: auto;
  padding: 1.5rem;
  background: var(--paper);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: 0 1rem 3rem rgba(28, 25, 23, 0.22);
}

.project-form-modal__content h2 {
  margin: 0 2.5rem 1.25rem 0;
  font-family: "Space Grotesk", -apple-system, sans-serif;
  font-size: 1.5rem;
}

.project-form-modal__close {
  position: absolute;
  top: 1rem;
  right: 1rem;
  border: 0;
  padding: 0;
  background: transparent;
  color: var(--text-muted);
  font-size: 1.8rem;
  line-height: 1;
  cursor: pointer;
}

```

```

}

.project-form-modal__actions {
  display: flex;
  justify-content: flex-end;
  gap: 0.75rem;
  margin-top: 1.5rem;
}

@media (max-width: 600px) {
  .project-form-modal {
    align-items: flex-end;
    padding: 0;
  }

  .project-form-modal__content {
    width: 100%;
    max-height: 90vh;
    padding: 1.25rem;
    border-radius: var(--radius) var(--radius) 0 0;
  }

  .project-form-modal__actions {
    flex-direction: column-reverse;
  }

  .project-form-modal__actions .button {
    width: 100%;
    text-align: center;
  }
}

```

Jalankan server, *login* sebagai *superuser*, lalu buka halaman `/projects/`. Tombol **Tambah Proyek** sekarang membuka form di atas halaman yang sama. Setelah tombol *submit* ditekan, form masih menggunakan *endpoint* lama `create_project`. Pada bagian selanjutnya, pengiriman tersebut akan kita ubah menjadi AJAX tanpa perlu mengubah struktur modalnya.

Menambahkan Data Proyek dengan AJAX

Sekarang halaman Projects sudah bisa *membaca* data lewat AJAX. Langkah berikutnya adalah *menulis* data, yaitu menambahkan proyek baru dari modal tanpa pindah halaman dan tanpa *reload* browser. Setelah proyek tersimpan, daftar proyek diperbarui dengan memanggil ulang `fetchProjects()`, dan hasilnya dikabarkan lewat toast yang sudah kita buat sebelumnya.

i Catatan

Bagian ini melanjutkan modal dari bagian sebelumnya. Pastikan modal kamu memiliki `<form id="project-form">` yang berisi *input* dengan atribut *name* sesuai *field* `ProjectForm` (`title`, `description`, `tech_stack`, `project_url`, `project_image_url`) beserta sebuah `<button type="submit">`, dan terdapat fungsi JavaScript `closeProjectModal()` untuk menutup modal.

Langkah 1: Membuat View `create_project_ajax`

Buka `main/views.py`, lalu tambahkan fungsi berikut:

```
from django.views.decorators.http import require_POST

...

@require_POST
def create_project_ajax(request):
    if not request.user.is_superuser:
        return JsonResponse(
            {"message": "Hanya pemilik portofolio yang dapat menambahkan proyek."},
            status=403,
        )

    form = ProjectForm(request.POST)
    if form.is_valid():
        project = form.save()
        return JsonResponse(
            {"message": "Proyek berhasil ditambahkan.", "pk": str(project.id)},
            status=201,
        )

    return JsonResponse({"errors": form.errors.get_json_data()}, status=400)
```

Penjelasan Kode

- `@require_POST` membuat *view* ini hanya menerima metode `POST`. Permintaan dengan metode lain langsung dibalas **405 Method Not Allowed**.
- Pemeriksaan `is_superuser` tetap wajib, sama seperti pada `create_project` di Tutorial 04. Menyembunyikan tombol di *template* tidak mencegah siapa pun memanggil *endpoint* ini langsung dengan `fetch`.
- Di sini kita **tidak** memakai `@login_required`. Dekorator itu membalas pengunjung yang belum *login* dengan `redirect` ke halaman *login*, dan `fetch` akan mengikuti `redirect` tersebut lalu menerima halaman HTML *login* dengan status 200 sehingga JavaScript kita tidak bisa

mengenalinya. Karena `AnonymousUser` juga memiliki `is_superuser` bernilai `False`, satu pemeriksaan di atas sudah menolak baik pengunjung yang belum *login* maupun pengguna biasa dengan respons JSON **403** yang mudah dibaca JavaScript.

- Kita memakai kembali `ProjectForm` alih-alih membuat objek `Project` langsung dari `request.POST`. Dengan begitu, semua validasi yang sudah ada (*field* wajib, panjang maksimum, format URL) tetap berlaku untuk permintaan AJAX.
- Status **201 Created** menandakan data baru berhasil dibuat, sedangkan **400 Bad Request** dikirim bersama pesan kesalahan tiap *field* dari `form.errors.get_json_data()`, misalnya `{"title": [{"message": "This field is required.", "code": "required"}]}`.

Langkah 2: Menambahkan Routing

Buka `main/urls.py`, impor fungsi `create_project_ajax`, lalu tambahkan *path* berikut ke dalam `urlpatterns`:

```
path("projects/add-ajax/", create_project_ajax, name="create_project_ajax"),
```

Langkah 3: Mengirim Data Form dengan Fetch API

Buka `project.html`, lalu tambahkan kode berikut ke dalam blok `<script>` yang sama dengan `fetchProjects`, tepat **sebelum** baris `fetchProjects(searchInput.value.trim());` di bagian paling bawah:

```
const CREATE_PROJECT_ENDPOINT = `${url 'main:create_project_ajax' %}`;
const projectForm = document.getElementById('project-form');

// Membaca nilai cookie, digunakan untuk mengambil token CSRF
function getCookie(name) {
    let cookieValue = null;
    if (document.cookie && document.cookie !== '') {
        const cookies = document.cookie.split(';');
        for (let i = 0; i < cookies.length; i++) {
            const cookie = cookies[i].trim();
            if (cookie.substring(0, name.length + 1) === (name + '=')) {
                cookieValue = decodeURIComponent(cookie.substring(name.length
+ 1));
                break;
            }
        }
    }
    return cookieValue;
}

// Mengirim data form ke server
async function addProject(event) {
```

```

    event.preventDefault();

    const submitButton = projectForm.querySelector('button[type="submit"]');
    submitButton.disabled = true;

    try {
        const response = await fetch(CREATE_PROJECT_ENDPOINT, {
            method: 'POST',
            headers: { 'X-CSRFToken': getCookie('csrftoken') },
            body: new FormData(projectForm),
        });
        const result = await response.json().catch(() => ({}));

        if (response.ok) {
            projectForm.reset();
            closeProjectModal();
            showToast('Berhasil', 'Proyek baru berhasil ditambahkan!',
'success');
            fetchProjects(searchInput.value.trim());
        } else {
            const errorMessages = result.errors
? Object.values(result.errors).flat().map(error => error.message)
: [result.message || `Terjadi kesalahan (status
${response.status})`];
            showToast('Gagal menambahkan proyek', errorMessages.join('
'), 'error');
        }
    } catch (error) {
        console.error('Error adding project:', error);
        showToast('Gagal menambahkan proyek', 'Tidak dapat terhubung ke
server. Silakan coba lagi.', 'error');
    } finally {
        submitButton.disabled = false;
    }
}

if (projectForm) {
    projectForm.addEventListener('submit', addProject);
}

```

Penjelasan Kode

- Modal hanya disertakan untuk *superuser* sehingga bagi pengunjung lain `projectForm` bernilai `null`. Pemeriksaan `if (projectForm)` mencegah `addEventListener` memicu *error* yang akan

menghentikan sisa skrip, termasuk pemanggilan `fetchProjects()`. Dengan begitu, daftar proyek tetap tampil untuk semua pengunjung.

- `getCookie` adalah fungsi pembaca *cookie* yang sudah kita bahas secara konseptual di Tutorial 04. Kini kita benar-benar menggunakannya untuk mengambil token dari *cookie* `csrftoken` dan mengirimkannya lewat *header* `X-CSRFToken`. `CsrfViewMiddleware` menerima token dari *header* ini atau dari *field* `csrfmiddlewaretoken` di dalam form (yang juga ikut terkirim lewat `FormData`). Permintaan tanpa salah satunya ditolak dengan status **403 Forbidden**.
- `new FormData(projectForm)` mengumpulkan seluruh isi *input* di dalam form berdasarkan atribut *name*-nya. Data tersebut dikirim dengan format `multipart/form-data`, dan Django tetap membacanya lewat `request.POST` sehingga data dapat divalidasi oleh `ProjectForm`.
- `event.preventDefault()` mencegah browser mengirim form dengan cara biasa (yang akan *reload* halaman). Pengiriman diambil alih sepenuhnya oleh `fetch`.
- Tombol *submit* dinonaktifkan selama permintaan berjalan agar proyek yang sama tidak tersimpan dua kali ketika tombol diklik berulang kali. Blok `finally` mengaktifkannya kembali, baik permintaan berhasil maupun gagal.
- `response.ok` bernilai `true` untuk status 2xx. Jika berhasil, form dikosongkan, modal ditutup, toast sukses ditampilkan, dan `fetchProjects()` dipanggil ulang dengan kata kunci pencarian yang sedang aktif sehingga proyek baru langsung muncul jika sesuai dengan kata kunci tersebut.
- Jika gagal, pesan kesalahan dari server ditampilkan lewat toast. `.catch(() => ({}))` pada `response.json()` menangani respons yang bukan JSON (misalnya halaman HTML 403 dari kegagalan CSRF) agar tetap ditampilkan sebagai pesan dengan kode statusnya.

Langkah 4: Pengujian

1. Jalankan server, lalu *login* menggunakan akun *superuser*.
2. Buka `/projects/`, buka modal tambah proyek, isi form, lalu tekan tombol *submit*. Modal akan tertutup, toast sukses muncul, dan proyek baru tampil di daftar tanpa *reload*.
3. Validasi bawaan browser (`required` dan `type="url"`) sudah memblokir judul kosong atau URL yang formatnya salah sebelum *submit handler* kita berjalan. Untuk menguji validasi server, isi **Nama Proyek** hanya dengan spasi (Django membuang spasi tersebut lalu membalas “This field is required.”) atau isi **URL Proyek** dengan `javascript:alert(1)` (browser menerimanya, tetapi Django menolaknya). Modal tetap terbuka dan toast dengan garis merah menampilkan pesan kesalahan dari server.
4. Buka tab **Network** pada DevTools (F12). Kamu akan melihat permintaan `POST` ke `/projects/add-ajax/` dengan status `201` (atau `400` saat input tidak valid), lalu disusul permintaan `GET` ke `/api/projects/`. Pada bagian **Request Headers**, perhatikan *header* `X-CSRFToken` yang ikut terkirim.

Melindungi Aplikasi dari XSS

Cross-Site Scripting (XSS) adalah serangan ketika penyerang berhasil menyisipkan kode JavaScript miliknya ke dalam halaman web yang kemudian dijalankan di browser pengguna lain. Salah satu jenisnya adalah *stored XSS*, yaitu ketika kode berbahaya disimpan ke database (misalnya sebagai judul proyek) lalu ikut dijalankan setiap kali data tersebut ditampilkan.

Di Tutorial 03 dan 04, aplikasi kita sebenarnya sudah terlindungi dari sebagian besar XSS tanpa kita sadari. *Template* Django melakukan **auto-escaping** pada setiap { variabel }. Karakter seperti < dan > diubah menjadi < dan > sehingga browser menampilkannya sebagai teks biasa, bukan sebagai tag HTML.

Perlindungan itu hilang ketika kita pindah ke AJAX. Pada `buildProjectCardElement`, data dari JSON disisipkan ke dalam *template literal* lalu dipasang lewat `innerHTML`. Tidak ada lagi Django yang melakukan *escaping* sehingga browser akan memperlakukan setiap tag HTML di dalam data sebagai kode sungguhan.

Langkah 1: Membuktikan Celah XSS

⚠ Peringatan

Lakukan percobaan ini hanya pada aplikasi milikmu sendiri di lingkungan lokal.

1. *Login* sebagai *superuser*, lalu tambahkan proyek baru lewat modal dengan **Nama Proyek** berikut:

```

```

2. Setelah proyek tersimpan, daftar proyek dimuat ulang dan sebuah *alert* bertuliskan **XSS!** akan muncul. *Alert* yang sama juga muncul untuk **setiap pengunjung** yang membuka `/projects/`, termasuk yang belum *login*.

Browser mencoba memuat gambar dari alamat `x`, gagal, lalu menjalankan isi atribut `onerror`. Kali ini isinya hanya `alert`, tetapi penyerang bisa menulis JavaScript apa pun. Ingat bahwa *cookie csrftoken* bisa dibaca oleh JavaScript. Artinya, kode yang disisipkan dapat membaca token itu lalu mengirim permintaan *POST* atas nama korban, misalnya memberi *star* atau, jika korbannya pemilik portofolio, menghapus proyek. Perlindungan CSRF dari Tutorial 04 tidak lagi efektif ketika penyerang sudah bisa menjalankan kode di halaman kita sendiri.

i Catatan

Pada aplikasi ini hanya pemilik portofolio yang dapat menambah proyek sehingga risikonya tampak kecil. Namun, pada banyak aplikasi (termasuk tugasmu nanti) pengguna biasa juga dapat mengirim data, dan akun pemilik pun bisa saja dibobol. Karena itu, **jangan pernah memercayai data yang akan ditampilkan**, dari mana pun asalnya.

Langkah 2: Melakukan *Escaping* di JavaScript

Perbaikan utamanya adalah mengubah karakter khusus HTML menjadi *entity* sebelum data dirakit menjadi HTML, persis seperti yang tadinya dikerjakan Django secara otomatis. Tambahkan fungsi berikut di dalam `<script>` pada `project.html`, di atas `buildProjectCardElement`:

```
// Mengubah karakter khusus HTML menjadi entity agar ditampilkan sebagai teks
function escapeHtml(value) {
  return String(value ?? '')
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;');
}
```

Kemudian, bungkus **setiap** nilai teks dari JSON yang disisipkan ke HTML dengan `escapeHtml`. Perbarui bagian-bagian berikut di dalam `buildProjectCardElement`:

```
const imageHtml = project.project_image_url
  ? ``
  : '';

const urlHtml = project.project_url
  ? `<a href="${escapeHtml(project.project_url)}" class="button">Lihat
    Project</a>`
  : '';

...

const starTitle = project.star_count > 0
  ? `Dibintangi oleh ${escapeHtml(project.starred_by_names)}`
  : "Jadilah yang pertama memberi star";

// Komponen card
const completeCardHtml = `
  ${imageHtml}
  <h2>${escapeHtml(project.title)}</h2>
  <span class="experience-category">${escapeHtml(project.tech_stack)}
</span>
  <p class="experience-description">${escapeHtml(project.description)}
</p>
  ...
`;
```

Muat ulang `/projects/`. *Alert* tidak lagi muncul, dan judul proyek tadi kini tampil apa adanya sebagai teks ``. Hapus proyek uji tersebut dengan tombol **Hapus** sebelum melanjutkan.

Penjelasan Kode

- `escapeHtml` mengubah lima karakter yang sama dengan yang di-*escape* oleh *template* Django. & harus diganti paling awal agar & pada hasil penggantian karakter lain tidak ikut diubah lagi.
- Tanda kutip " dan ' juga wajib di-*escape* karena sebagian data disisipkan **di dalam atribut**, seperti `alt="Gambar ${...}"` dan `title="${starTitle}"`. Tanpa itu, judul seperti `onmouseover="alert(1)` dapat menutup atribut `alt` lebih awal lalu menambahkan atribut *event handler* baru, meskipun tidak mengandung `<` sama sekali.
- `?? ''` mengubah nilai `null` atau `undefined` menjadi *string* kosong agar tidak tampil sebagai teks `"null"`.
- *Escaping* tidak menghalangi URL berbahaya seperti `javascript:alert(1)` di atribut `href` karena URL tersebut tidak mengandung karakter khusus HTML. Celah ini sudah tertutup di sisi server karena `URLField` pada `ProjectForm` menolak skema seperti `javascript:..`. Inilah alasan kita memakai `ProjectForm` di `create_project_ajax` pada bagian sebelumnya.

Tip

Alternatif lain adalah membangun elemen dengan `document.createElement()` lalu mengisi teksnya lewat `textContent`. Properti `textContent` tidak pernah menafsirkan isinya sebagai HTML. Fungsi `showToast` yang kita buat sebelumnya sudah mengisi judul dan pesan toast lewat `textContent` sehingga aman menampilkan pesan kesalahan dari server. Jika suatu saat kamu memang perlu menampilkan HTML dari pengguna (misalnya teks berformat), gunakan *library sanitizer* seperti `DOMPurify` alih-alih menulis penyaring sendiri.

Langkah 3: Membersihkan Input di Server

Sebagai lapisan pertahanan kedua, kita juga dapat membuang tag HTML sejak data masuk. Buka `main/forms.py`, lalu tambahkan *method* berikut ke dalam kelas `ProjectForm` (sejajar dengan kelas `Meta`):

```
from django.core.exceptions import ValidationError
from django.utils.html import strip_tags

...

class ProjectForm(ModelForm):
    class Meta:
        ...

    def clean_title(self):
        title = strip_tags(self.cleaned_data["title"]).strip()
        if not title:
            raise ValidationError("Nama proyek tidak boleh hanya berisi tag HTML.")
        return title
```

```
def clean_tech_stack(self):
    return strip_tags(self.cleaned_data["tech_stack"]).strip()

def clean_description(self):
    return strip_tags(self.cleaned_data["description"]).strip()
```

Penjelasan Kode

- *Method* dengan pola nama `clean_<nama_field>` dijalankan otomatis oleh Django saat `form.is_valid()` dipanggil, setelah validasi bawaan *field* tersebut lolos. Nilai yang dikembalikan menggantikan isi `cleaned_data` untuk *field* itu, dan nilai inilah yang disimpan oleh `form.save()`.
- `strip_tags` menghapus semua tag HTML dari teks sehingga Halo `<b>dunia</b>` disimpan sebagai Halo dunia.
- `clean_title` menolak judul yang menjadi kosong setelah tag dihapus, misalnya *payload* `<img ...>` tadi, dan pesan kesalahannya akan tampil di toast.
- Karena `create_project` dan `create_project_ajax` sama-sama memakai `ProjectForm`, pembersihan ini berlaku untuk kedua jalur penambahan proyek sekaligus.

! Penting

`strip_tags` **bukan pengganti** *escaping* di Langkah 2. Dokumentasi Django sendiri menyatakan bahwa hasil `strip_tags` tidak dijamin aman untuk ditampilkan sebagai HTML. Selain itu, teks seperti `List<String>` ikut terpotong menjadi `List`, dan pembersihan ini hanya berlaku untuk data baru. Proyek berbahaya yang telanjur tersimpan di database tetap ada. Karena itu, *escaping* saat menampilkan data adalah pertahanan utama, sedangkan pembersihan input adalah lapisan tambahan.

Coba tambahkan kembali proyek dengan judul ``. Kali ini server menolaknya dan toast dengan garis merah menampilkan pesan kesalahan dari `clean_title`.

! Menghubungkan ke Proyek Kamu

Setiap kali kamu menampilkan data lewat AJAX di bagian lain portofolionmu (misalnya pengalaman, pendidikan atau sertifikasi), terapkan pola yang sama. Bungkus setiap nilai yang disisipkan ke `innerHTML` dengan `escapeHtml`, gunakan `ModelForm` beserta *method* `clean_<field>` untuk memvalidasi input di *endpoint* AJAX, dan tetap periksa hak akses di dalam *view*, bukan hanya di *template*.

Pengumpulan

Tutorial ini dikumpulkan lewat slot submisi yang disediakan di SCELE, dalam bentuk **tautan ke commit** (bukan sekadar tautan repositori) di GitHub yang menunjukkan hasil akhir Tutorial ini, di-*push* **sebelum tenggat waktu** di atas. Commit yang di-*push* setelah tenggat waktu tidak akan diterima, sehingga tutorial ini dianggap belum selesai dan Individual Assignment terkait tidak akan dinilai. Pastikan juga repositori GitHub kamu bersifat **publik**, supaya asisten dosen bisa mengaksesnya.

Referensi Tambahan

- Tutorial 5 PBP Ganjil 2025/2026